

utfcode

utf8.sty 3.10 UTF-8 input encoding 13.06.2000

scanner for code UTF-8 installed.

Tanker Freight & TCE Dashboard

Financial Modeling of Maritime Economics, Worldscale Rates, and Interactive Sensitivity Analysis

Alexis Gaire

gairealexis@gmail.com

GitHub Repository: <https://github.com/AlexisGpro/tanker-freight-tce-dashboard>

September 24, 2026

Abstract

This technical report presents the architecture, financial modeling, and software implementation of the **Tanker Freight & TCE Dashboard**, a quantitative decision-support tool engineered for crude oil chartering and trading desks (e.g., TotalEnergies / BxT Trading). The application normalizes voyage charter revenues across major tanker vessel classes (VLCC, Suezmax, Aframax) by computing the daily Time Charter Equivalent (*TCE*). Incorporating Worldscale (*WS*) benchmark conversions, bunker fuel consumption dynamics, a cash-flow waterfall breakdown, and a multi-variable sensitivity matrix, this tool enables real-time profitability evaluation under volatile market conditions.

1 Introduction & Business Context

In energy trading and chartering operations, comparing the profitability of spot voyage charters against time charter opportunities requires a normalized financial metric. While spot rates are quoted in lump sums or per metric ton (\$/mt), operating expenses such as port costs and bunker fuel expenditures are borne by the shipowner.

The **Time Charter Equivalent (TCE)** serves as the industry-standard KPI, converting net voyage cash flows into an equivalent daily hire rate (\$/day). Key objectives of this project include:

1. Modeling voyage economics across key benchmark routes (TD3C, TD20, TD7) and vessel classes.
2. Integrating Worldscale (WS) flat rate conversions into absolute freight rates.
3. Developing an interactive Streamlit interface featuring Plotly waterfall cost breakdowns and sensitivity heatmaps.

2 Financial & Mathematical Modeling

2.1 Worldscale Rate Conversion

Freight rates in the tanker market are commonly quoted in Worldscale (WS) points. Given a market quote WS_{points} and an annual benchmark Flat Rate FR_{flat} (\$/mt), the effective freight rate R_{freight} in USD per metric ton is computed as:

$$R_{\text{freight}} = \left(\frac{WS_{\text{points}}}{100} \right) \times FR_{\text{flat}} \quad (1)$$

Gross revenue G_{revenue} is derived from the loaded cargo quantity Q_{cargo} (metric tons):

$$G_{\text{revenue}} = R_{\text{freight}} \times Q_{\text{cargo}} \quad (2)$$

2.2 Voyage Duration and Bunker Expenditures

Voyage duration T_{total} (days) accounts for a round-trip sea passage at speed V_{knots} over distance D_{nm} (nautical miles), plus total port and operational waiting days T_{port} :

$$T_{\text{sea}} = \frac{2 \times D_{\text{nm}}}{24 \times V_{\text{knots}}}, \quad T_{\text{total}} = T_{\text{sea}} + T_{\text{port}} \quad (3)$$

Total bunker fuel expenditure C_{bunker} is calculated using sea fuel consumption F_{sea} (tons/day), port consumption F_{port} (tons/day), and fuel price P_{fuel} (\$/ton):

$$C_{\text{bunker}} = (T_{\text{sea}} \times F_{\text{sea}} + T_{\text{port}} \times F_{\text{port}}) \times P_{\text{fuel}} \quad (4)$$

2.3 Time Charter Equivalent (TCE) Formulation

Deducting port disbursements C_{port} and bunker fuel costs C_{bunker} yields the Net Voyage Profit N_{profit} . The net TCE rate (\$/day) is expressed as:

$$N_{\text{profit}} = G_{\text{revenue}} - C_{\text{port}} - C_{\text{bunker}} \quad (5)$$

$$\text{TCE} = \frac{N_{\text{profit}}}{T_{\text{total}}} = \frac{(R_{\text{freight}} \times Q_{\text{cargo}}) - C_{\text{port}} - C_{\text{bunker}}}{T_{\text{sea}} + T_{\text{port}}} \quad (6)$$

3 Software Architecture & Implementation

The architecture enforces a strict separation between core domain logic and the presentation layer:

- `calculator.py`: Pure Python module handling specs, WS conversion, bunker costs, and sensitivity grid generation.
- `app.py`: User interface built with `Streamlit` and interactive charts via `Plotly`.

3.1 Core Calculation Engine (`calculator.py`)

```

1 import numpy as np
2 import pandas as pd
3
4 def calculate_bunker_cost(days_at_sea: float, sea_cons: float,
5                           days_in_port: float, port_cons: float,
6                           fuel_price: float) -> float:
7     total_fuel_tons = (days_at_sea * sea_cons) + (days_in_port *
8     port_cons)
9     return total_fuel_tons * fuel_price
10
11 def calculate_tce(freight_rate: float, cargo_qty: float,
12                  port_costs: float, bunker_cost: float,
13                  total_days: float) -> float:
14     gross_revenue = freight_rate * cargo_qty
15     net_revenue = gross_revenue - port_costs - bunker_cost
16     if total_days <= 0:

```

```
16         return 0.0
17     return net_revenue / total_days
```

Listing 1: TCE calculation and sensitivity engine

4 Visual Analytics & Dashboard Features

The interactive dashboard provides chartering managers and market analysts with:

- **Financial KPIs:** Instantaneous display of Gross Revenue, Bunker Cost, Net Profit, and Net TCE/day.
- **Cash Flow Waterfall:** Plotly visualization illustrating revenue degradation through port fees and fuel expenses down to net profit.
- **Sensitivity Heatmap:** Grid evaluating TCE output variations across a range of freight rates ($\pm 30\%$) and fuel prices ($\pm 30\%$).

5 Conclusion & Future Extensions

This project provides a robust foundation for evaluating tanker voyage economics. Future improvements include integrating live bunker price APIs and connecting historical fixture datasets for automated backtesting.